

NAME - KRISH CHANDRASHEKHAR PACHODE ROLL NO. - B-18 BATCH - AIDS (SY-B)

AIM - Building a Simple Data Science Pipeline: Integrate all learned techniques in previous labs into a complete workflow.

THEORY -

1. What is a Data Science pipeline? Explain its key stages with examples.

A **Data Science pipeline** is a sequence of steps used to collect, process, analyze, and model data to obtain useful insights.

Key Stages:

- **Data Collection:** Gathering data (e.g., CSV file, database)
- **Data Cleaning:** Handling missing values, removing duplicates
- **EDA (Exploratory Data Analysis):** Understanding data using graphs and statistics
- **Feature Engineering:** Creating new useful features
- **Model Building:** Applying machine learning algorithms
- **Evaluation:** Checking model performance

□ Example: Predicting student marks based on study hours.

2. What is Exploratory Data Analysis (EDA)? How do univariate and bivariate analyses help in understanding the dataset?

EDA is the process of analyzing data to understand its structure, patterns, and relationships before applying models.

- **Univariate Analysis:** Studies one variable at a time (e.g., histogram of marks)
- **Bivariate Analysis:** Studies relationship between two variables (e.g., scatter plot of hours vs marks)

□ These help in identifying trends, outliers, and relationships.

3. Differentiate between Label Encoding and OneHot Encoding. When should each be used?

Feature	Label Encoding	OneHot Encoding
Representation	Assigns numeric values	Creates binary columns
Example	Red=0, Blue=1, Green=2	Red(1,0,0), Blue(0,1,0)
Use Case	Ordinal data (with order)	Nominal data (no order)

□ Use Label Encoding for ordered data, OneHot Encoding for categorical data without order.

4. Why is feature scaling required? Compare normalization and standardization.

Feature scaling ensures all variables are on the same scale, improving model performance.

- **Normalization:** Scales values between 0 and 1
- **Standardization:** Transforms data to mean = 0 and standard deviation = 1

□ Scaling is important for algorithms like KNN, SVM, and gradient descent.

5. What is feature engineering? Explain how creating new features can improve model performance.

Feature engineering is the process of creating new features from existing data to improve model performance.

Example:

- Creating "Total Marks" from subject-wise marks
- Extracting "Age" from date of birth

□ Better features help models learn patterns more effectively, improving accuracy.

Conclusion

In this assignment, we studied the concept of a data science pipeline and its stages. We learned about EDA, encoding techniques, feature scaling, and feature engineering. These steps are essential for building effective and accurate machine learning models.

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, OneHotEncoder,
StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# -----
# STEP 1: Data Ingestion
# -----
data = {
    "Hours_Studied": [1,2,3,4,5,6,7,8,9,10],
```

```

        "Sleep_Hours": [7,6,6,5,5,6,7,8,7,6],
        "Gender":
["Male", "Female", "Male", "Female", "Male", "Female", "Male", "Female", "Male", "Female"],
        "Marks": [30,35,40,50,55,60,70,75,80,90]
    }

df = pd.DataFrame(data)
print("Dataset:\n", df.head())

# -----
# STEP 2: Data Cleaning
# -----
print("\nMissing Values:\n", df.isnull().sum())

# -----
# STEP 3: EDA
# -----
# Univariate
plt.figure(figsize=(6,4))
plt.hist(df["Marks"])
plt.title("Distribution of Marks")
plt.tight_layout()
plt.show()

# Bivariate
plt.figure(figsize=(6,4))
plt.scatter(df["Hours_Studied"], df["Marks"])
plt.title("Hours Studied vs Marks")
plt.xlabel("Hours")
plt.ylabel("Marks")
plt.tight_layout()
plt.show()

# -----
# STEP 4: Encoding
# -----
# Label Encoding
le = LabelEncoder()
df["Gender_Label"] = le.fit_transform(df["Gender"])

# OneHot Encoding
df = pd.get_dummies(df, columns=["Gender"])

print("\nAfter Encoding:\n", df.head())

# -----
# STEP 5: Feature Scaling
# -----
scaler = StandardScaler()

```

```

df[["Hours_Studied", "Sleep_Hours"]] =
scaler.fit_transform(df[["Hours_Studied", "Sleep_Hours"]])

# -----
# STEP 6: Feature & Target
# -----
X = df.drop("Marks", axis=1)
y = df["Marks"]

# -----
# STEP 7: Train-Test Split
# -----
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)

# -----
# STEP 8: Model Training
# -----
model = LinearRegression()
model.fit(X_train, y_train)

# -----
# STEP 9: Prediction
# -----
y_pred = model.predict(X_test)

# -----
# STEP 10: Evaluation
# -----
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\nModel Performance:")
print(f"MSE: {mse:.2f}")
print(f"R2 Score: {r2:.4f}")

```

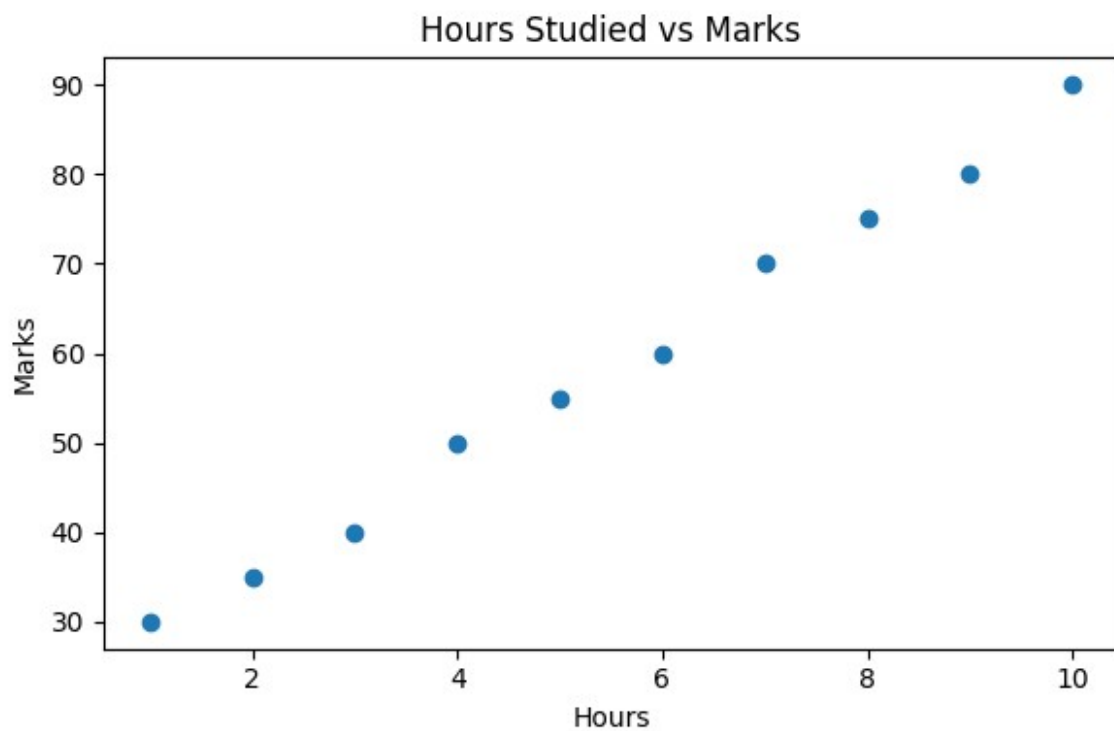
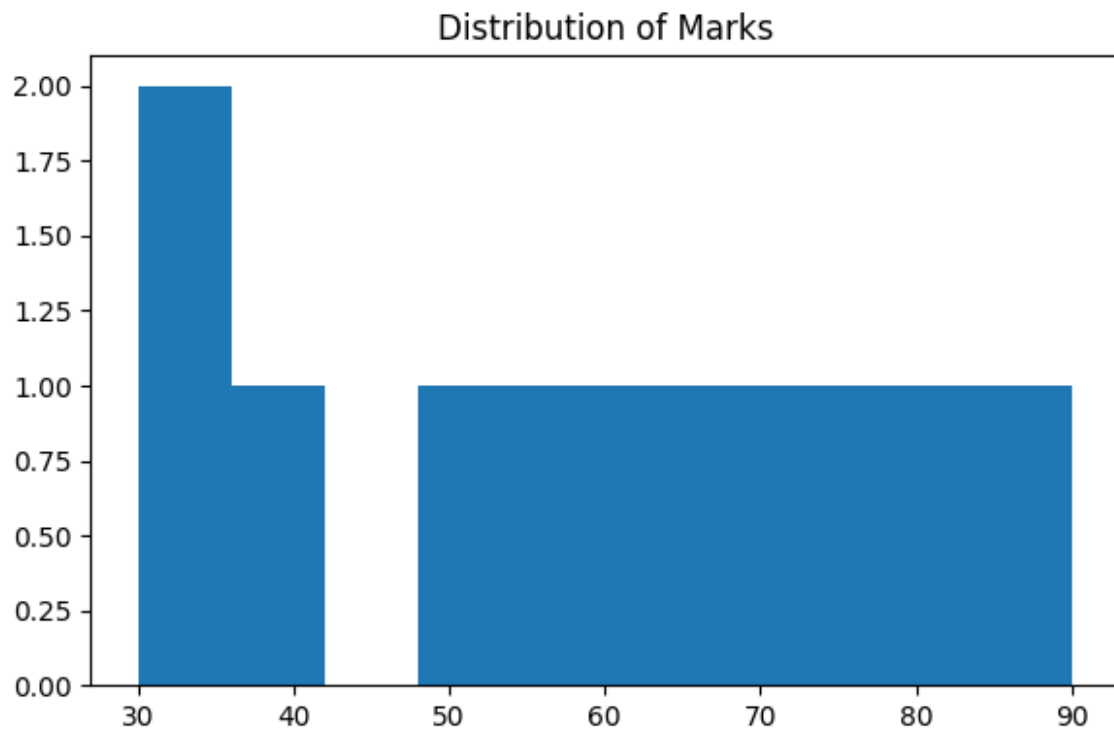
Dataset:

	Hours_Studied	Sleep_Hours	Gender	Marks
0	1	7	Male	30
1	2	6	Female	35
2	3	6	Male	40
3	4	5	Female	50
4	5	5	Male	55

Missing Values:

Hours_Studied	0
Sleep_Hours	0
Gender	0
Marks	0

dtype: int64



After Encoding:

Hours_Studied	Sleep_Hours	Marks	Gender_Label	Gender_Female
Gender_Male				

0	1	7	30	1	False
True					
1	2	6	35	0	True
False					
2	3	6	40	1	False
True					
3	4	5	50	0	True
False					
4	5	5	55	1	False
True					

Model Performance:

MSE: 8.44

R² Score: 0.9625